Guide Développeur Complet

Modèle Machine Learning Anti-Triche pour Code Source

Comment créer, développer et déployer ce système de A à Z

Rapport Technique Détaillé

26 février 2026

Pour qui est ce guide ?

Ce document est destiné à un **développeur** qui souhaite comprendre et reproduire entièrement un système de détection de code généré par IA. Après lecture, vous serez capable de créer votre propre version de ce modèle, avec vos propre données et vos propres critères, et de le déployer en production sur n'importe quelle plateforme cloud.

Stack Technologique Utilisée :

Couche	Technologie	Rôle
Backend / API	Python + FastAPI	Serveur web, routes REST
ML / IA	Scikit-Learn (RandomForest)	Modèle de classification
Analyse Code	ast (Python Standard)	Parsing syntaxique du code
Frontend	HTML + CSS + JavaScript	Interface visuelle utilisateur
Hébergement	Hugging Face Spaces	Serveur cloud gratuit
Persitance	Git / JSONL	Stockage des prédictions

Table des matières

1 Vue d'Ensemble : Comprendre la Philosophie **3**
1.1 Que fait concrètement ce modèle ? . . . 3
1.2 Pourquoi un RandomForest et pas un LLM ? . . . 3

2 Étape 1 : Préparer l'Environnement de Développement **3**
2.1 Prérequis . . . 3
2.2 Installation des dépendances . . . 3
2.3 Structure du projet à créer . . . 3

3 Étape 2 : L'Extraction de Features (`features.py`) **4**
3.1 Les 9 Features Utilisées et Leur Justification . . . 4
3.2 Le Code Complet de `features.py` . . . 4

4 Étape 3 : Créer le Dataset d'Entraînement **6**
4.1 Principe . . . 6
4.2 Où trouver des données ? . . . 6
4.3 Format du Dataset . . . 6
4.4 Lancer l'entraînement initial . . . 8

5 Étape 4 : Créer l'API avec FastAPI (`api.py`) **8**
5.1 Structure de Base de l'API . . . 8
5.2 Intégration de l'Analyse GitHub . . . 9

6 Étape 5 : Apprentissage Continu **10**
6.1 Principe de la Boucle d'Apprentissage . . . 10
6.2 Script de Ré-entraînement (`training.py`) . . . 10

7 Étape 6 : Déploiement sur Hugging Face Spaces **11**
7.1 Pourquoi Hugging Face Spaces ? . . . 11
7.2 Procédure de Déploiement . . . 11
7.2.1 1. Créer un Space . . . 12
7.2.2 2. Fichier de configuration Hugging Face . . . 12
7.2.3 3. Dockerfile . . . 12
7.2.4 4. Ajouter les Secrets (Variables d'environnement) . . . 12
7.2.5 5. Déployer . . . 12

8 Ajouter des Features Supplémentaires **13**
8.1 Features Avancées Suggérées . . . 13

9 Conclusion et Conseils **13**

1 Vue d'Ensemble : Comprendre la Philosophie

1.1 Que fait concrètement ce modèle ?

Le modèle analyse du **code source** (Python, Java, JavaScript, etc.) et retourne un score de 0 à 100% représentant la probabilité que ce code ait été **généré par une IA** (ChatGPT, Copilot, etc.) plutôt qu'écrit à la main par un humain.

1.2 Pourquoi un RandomForest et pas un LLM ?

Une question importante pour un développeur est : pourquoi ne pas utiliser ChatGPT pour détecter ChatGPT ?

La réponse est la suivante :

— **Coût** : Les APIs de LLM (GPT-4, Claude) coûtent très cher à l'échelle (des dizaines d'euros par millier d'analyses).
— **Vitesse** : Un RandomForest répond en <5ms. Un LLM répond en 3 à 8 secondes.
— **Déterminisme** : Un LLM peut changer d'avis d'une invocation à l'autre. Un modèle ML donne toujours le même résultat pour la même entrée.
— **Contrôle** : On sait exactement *pourquoi* le modèle prend une décision (interprétabilité des features).

Le RandomForest est un **ensemble de 100 arbres de décision**. Chaque arbre "vote" pour une classe (Humain ou IA), et la majorité gagne. C'est robuste, rapide, et explicable.

2 Étape 1 : Préparer l'Environnement de Développement

2.1 Prérequis

— Python 3.10 ou supérieur
— `pip` (gestionnaire de paquets Python)
— Un compte GitHub (pour tester des dépôts réels)
— Un compte Hugging Face (pour le déploiement cloud gratuit)

2.2 Installation des dépendances

Créez un fichier `requirements.txt` avec le contenu suivant :

```
fastapi>=0.110.0
uvicorn>=0.29.0
scikit-learn>=1.4.0
numpy>=1.26.0
pydantic>=2.0.0
huggingface_hub>=0.22.0
requests>=2.31.0
python-multipart>=0.0.9
```

Listing 1 – requirements.txt

Puis installez :

```
pip install -r requirements.txt
```

2.3 Structure du projet à créer

Voici l'arborescence minimale pour que le projet fonctionne :

```
mon_projet_anticheat/
|-- src/
|   |-- __init__.py
|   |-- api.py           # Serveur FastAPI + toutes les routes
|   |-- features.py      # Extraction des caracteristiques du code
|   |-- model_ia.py      # Fonctions train/load/save du modele
|   |-- storage.py       # Sauvegarde des predictions et decisions
|   |-- training.py      # Script de re-entrainement
|   |-- templates/
|       |-- index.html   # Interface web
|-- models/
|   |-- model_v1.pkl     # Le premier modele entraine
|-- storage/
|   |-- predictions.jsonl
|   |-- human_decisions.jsonl
|-- scripts/
|   |-- train_initial_model.py
|-- config.py
|-- requirements.txt
```

Listing 2 – Arborescence du projet

3 Étape 2 : L'Extraction de Features (`features.py`)

C'est **LA partie la plus importante**. Si vos features sont mauvaises, votre modèle sera mauvais, peu importe l'algorithme utilisé. Le principe est de transformer du texte (code source) en un vecteur de nombres que l'algorithme peut comprendre.

3.1 Les 9 Features Utilisées et Leur Justification

Feature	Calcul	Pourquoi ça marche
`mean_line_length`	Longueur moyenne de toutes les lignes	Le code IA a des lignes typiquement plus longues
`std_line_length`	Écart-type de la longueur des lignes	Le code humain a des variations plus grandes
`indentation_variance`	Variance des espaces d'indentation	Humain = irrégulier, IA = parfait
`comment_ratio`	Nb commentaires / Nb lignes totales	IA sur-commente ou ne commente pas du tout
`max_nesting_depth`	Profondeur max d'imbrication (AST)	L'IA évite rarement les profondeurs excessives
`num_branches`	Nb de `if/for/while/with` (AST)	Densité de contrôle
`avg_identifier_length`	Longueur moy. des noms de variables	IA = noms très longs et explicites
`num_lines`	Nombre de lignes total	Simple mesure de taille
`empty_line_ratio`	Ratio lignes vides / total	IA aère parfaitement son code

3.2 Le Code Complet de `features.py`

```python
import re
import ast
from typing import List

def extract_line_length_stats(lines):
```

```
    """Longueur moyenne et ecart-type des lignes."""
    if not lines:
        return 0.0, 0.0
    lengths = [len(line) for line in lines]
    mean_ = sum(lengths) / len(lengths)
    var_ = sum((x - mean_)**2 for x in lengths) / len(lengths)
    return round(mean_, 2), round(var_**0.5, 2)

def extract_indentation_variance(lines):
    """Variance de l'indentation (irregularite = humain)."""
    indents = []
    for line in lines:
        stripped = line.lstrip()
        if stripped:
            indents.append(len(line) - len(stripped))
    if not indents:
        return 0.0
    mean_ = sum(indents) / len(indents)
    var_ = sum((x-mean_)**2 for x in indents) / len(indents)
    return round(var_**0.5, 2)

def extract_comment_ratio(lines):
    """Ratio commentaires / code."""
    code_lines = [l for l in lines if l.strip()]
    if not code_lines:
        return 0.0
    comments = sum(1 for l in code_lines
                   if l.strip().startswith("#") or '"""' in l)
    return round(comments / len(code_lines), 4)

def extract_complexity_indicators(code):
    """Analyse AST : profondeur, branches, longueur des noms."""
    try:
        tree = ast.parse(code)
    except SyntaxError:
        return 0, 0, 0.0

    name_lengths = []
    num_branches = 0
    for node in ast.walk(tree):
        if isinstance(node, (ast.For, ast.While, ast.If, ast.With)):
            num_branches += 1
        if isinstance(node, ast.Name):
            name_lengths.append(len(node.id))
        if isinstance(node, ast.FunctionDef):
            name_lengths.append(len(node.name))

    def depth_of(node, d=0):
        children = list(ast.iter_child_nodes(node))
        if not children:
            return d
        return max(depth_of(c, d+1) for c in children)

    max_depth = max((depth_of(n) for n in ast.walk(tree)), default=0)
    avg_name = sum(name_lengths)/len(name_lengths) if name_lengths else 0.0
    return max_depth, num_branches, round(avg_name, 2)

# Ordre fixe des features (DOIT etre le meme a l'entrainement et a la prediction)
FEATURE_NAMES = [
    "mean_line_length", "std_line_length", "indentation_variance",
    "comment_ratio", "max_nesting_depth", "num_branches",
    "avg_identifier_length", "num_lines", "empty_line_ratio",
```

```
]

def extract_all_features(code: str) -> dict:
    """Transforme du code texte en un dictionnaire de 9 nombres."""
    lines = code.strip().split("\n")
    mean_len, std_len = extract_line_length_stats(lines)
    return {
        "mean_line_length": mean_len,
        "std_line_length": std_len,
        "indentation_variance": extract_indentation_variance(lines),
        "comment_ratio": extract_comment_ratio(lines),
        "max_nesting_depth": extract_complexity_indicators(code)[0],
        "num_branches": extract_complexity_indicators(code)[1],
        "avg_identifier_length": extract_complexity_indicators(code)[2],
        "num_lines": len(lines),
        "empty_line_ratio": round(
            sum(1 for l in lines if not l.strip()) / len(lines), 4
        ) if lines else 0.0,
    }

def features_to_vector(features: dict) -> list:
    """Convertit le dict en liste ordonnee pour scikit-learn."""
    return [features.get(name, 0.0) for name in FEATURE_NAMES]
```

Listing 3 – src/features.py - Extraction des features

4 Étape 3 : Créer le Dataset d'Entraînement

4.1 Principe

Pour entraîner un RandomForest, il vous faut des **exemples étiquetés** :

— Des codes écrits par un **humain** (Label = 0)

— Des codes générés par une **IA** (Label = 1)

4.2 Où trouver des données ?

— **Codes Humains (Label 0)** : Dépôts GitHub anciens (avant 2020, avant ChatGPT), solutions d'exercices d'étudiants, vieux commits, StackOverflow.

— **Codes IA (Label 1)** : Demandez à ChatGPT / Copilot de résoudre 200 exercices différents. Les réponses seront vos exemples de Label 1.

4.3 Format du Dataset

Créez un script `scripts/train_initial_model.py` pour générer le premier modèle :

```
"""
Script d'entrainement initial du modele.
A executer UNE SEULE FOIS pour generer model_v1.pkl
"""
import json
import numpy as np
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
import pickle
from pathlib import Path

# Importez votre fonctions d'extraction
import sys
sys.path.insert(0, ".")
from src.features import extract_all_features, features_to_vector
```

```
16
17 # ============================================================
18 # Remplissez ici VOS exemples de code
19 # ============================================================
20 EXEMPLES_HUMAINS = [
21     """
22 def␣add(a,␣b):
23 ␣␣␣␣#␣addition␣simple
24 ␣␣␣␣resultat␣=␣a␣+␣b
25 ␣␣␣␣return␣resultat
26 ␣␣␣␣""",
27     """
28 x␣=␣[]
29 for␣i␣in␣range(10):
30 ␣␣␣␣if␣i%2␣==␣0:
31 ␣␣␣␣␣␣␣␣x.append(i)
32 print(x)
33 ␣␣␣␣""",
34     # Ajoutez ici 50 a 200 autres exemples de code humain...
35 ]
36
37 EXEMPLES_IA = [
38     """
39 def␣add_numbers(first_number:␣int,␣second_number:␣int)␣->␣int:
40 ␣␣␣␣\"\"\"
41 ␣␣␣␣Adds␣two␣numbers␣together␣and␣returns␣the␣result.
42 ␣␣␣␣Args:
43 ␣␣␣␣␣␣␣␣first_number:␣The␣first␣operand
44 ␣␣␣␣␣␣␣␣second_number:␣The␣second␣operand
45 ␣␣␣␣Returns:
46 ␣␣␣␣␣␣␣␣The␣sum␣of␣the␣two␣numbers
47 ␣␣␣␣\"\"\"
48 ␣␣␣␣return␣first_number␣+␣second_number
49 ␣␣␣␣""",
50     """
51 def␣process_list(input_list:␣list)␣->␣list:
52 ␣␣␣␣\"\"\"
53 ␣␣␣␣Filters␣even␣numbers␣from␣the␣input␣list.
54 ␣␣␣␣Args:
55 ␣␣␣␣␣␣␣␣input_list:␣A␣list␣of␣integers␣to␣process.
56 ␣␣␣␣Returns:
57 ␣␣␣␣␣␣␣␣A␣new␣list␣containing␣only␣even␣numbers.
58 ␣␣␣␣\"\"\"
59 ␣␣␣␣return␣[element␣for␣element␣in␣input_list␣if␣element␣%␣2␣==␣0]
60 ␣␣␣␣""",
61     # Ajoutez ici 50 a 200 exemples generes par ChatGPT...
62 ]
63
64 # Construire les vecteurs de features
65 X = []
66 y = []
67
68 for code in EXEMPLES_HUMAINS:
69     feats = extract_all_features(code)
70     X.append(features_to_vector(feats))
71     y.append(0)  # Label 0 = Humain
72
73 for code in EXEMPLES_IA:
74     feats = extract_all_features(code)
75     X.append(features_to_vector(feats))
76     y.append(1)  # Label 1 = IA
77
78 X_arr = np.array(X)
```

```
y_arr = np.array(y)

# Diviser en train (80%) et test (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X_arr, y_arr, test_size=0.2, random_state=42
)

# Entrainer le RandomForest
clf = RandomForestClassifier(
    n_estimators=100,    # 100 arbres de decision
    max_depth=10,        # Profondeur max de chaque arbre
    random_state=42      # Pour la reproducibilite
)
clf.fit(X_train, y_train)

# Evaluer la precision
accuracy = clf.score(X_test, y_test)
print(f"Precision du modele v1 : {accuracy * 100:.1f}%")

# Sauvegarder le modele
Path("models").mkdir(exist_ok=True)
with open("models/model_v1.pkl", "wb") as f:
    pickle.dump(clf, f)
print("Modele sauvegarde dans models/model_v1.pkl")
```

Listing 4 – scripts/train_initial_model.py

4.4 Lancer l'entraînement initial

```
python scripts/train_initial_model.py
# Sortie attendue :
# Precision du modele v1 : 91.7%
# Modele sauvegarde dans models/model_v1.pkl
```

5 Étape 4 : Créer l'API avec FastAPI (api.py)

5.1 Structure de Base de l'API

```
import pickle
import numpy as np
from fastapi import FastAPI
from fastapi.responses import HTMLResponse
from pydantic import BaseModel
from pathlib import Path

from src.features import extract_all_features, features_to_vector

app = FastAPI(title="Anti-Cheat API")

# MODELE GLOBAL : charge une seule fois au demarrage
_MODEL = None

def load_model_into_memory():
    global _MODEL
    model_path = Path("models/model_v1.pkl")
    if model_path.exists():
        with open(model_path, "rb") as f:
            _MODEL = pickle.load(f)

```

```python
@app.on_event("startup")
def startup():
    load_model_into_memory()

# ------ Modeles Pydantic (validation des donnees) ------
class AnalyzeRequest(BaseModel):
    code: str

class AnalyzeResponse(BaseModel):
    score_ia: float       # Entre 0.0 et 1.0
    verdict: str          # "OK", "SUSPECT" ou "TRICHE PROBABLE"

# ------ Route principale d'analyse ------
@app.post("/analyze", response_model=AnalyzeResponse)
def analyze(request: AnalyzeRequest):
    code = request.code

    # 1. Extraire les features du code
    features = extract_all_features(code)
    vector = features_to_vector(features)

    # 2. Predire avec le modele
    X = np.array([vector])
    # predict_proba retourne [[prob_classe_0, prob_classe_1]]
    proba = _MODEL.predict_proba(X)[0]
    score_ia = float(proba[1])  # probabilite d'etre de l'IA

    # 3. Calculer le verdict
    if score_ia < 0.40:
        verdict = "OK"
    elif score_ia < 0.65:
        verdict = "SUSPECT"
    else:
        verdict = "TRICHE␣PROBABLE"

    return AnalyzeResponse(score_ia=score_ia, verdict=verdict)

# ------ Page d'accueil ------
@app.get("/", response_class=HTMLResponse)
def root():
    return "<h1>API␣Anti-Triche␣en␣ligne!</h1>"
```

Listing 5 – src/api.py - Structure minimale de l'API

5.2 Intégration de l'Analyse GitHub

Pour analyser un dépôt GitHub entier automatiquement, on utilise l'API GitHub officielle :

```python
import urllib.request
import json
import os

class AnalyzeGitHubRequest(BaseModel):
    github_url: str

@app.post("/analyze_github")
def analyze_github(request: AnalyzeGitHubRequest):
    # 1. Parser l'URL (ex: https://github.com/user/repo)
    import re
    match = re.match(
        r"https://github\.com/([^/]+)/([^/\s?]+)", request.github_url
    )
    if not match:
```

```
        raise HTTPException(400, "URL invalide")
    owner, repo = match.group(1), match.group(2)

    # 2. Appeler l'API GitHub (avec token si disponible)
    github_token = os.environ.get("GITHUB_TOKEN")
    headers = {"Accept": "application/vnd.github.v3+json",
               "User-Agent": "AntiCheat-API"}
    if github_token:
        headers["Authorization"] = f"token {github_token}"

    # 3. Recuperer la liste des fichiers
    url = f"https://api.github.com/repos/{owner}/{repo}/contents/"
    req = urllib.request.Request(url, headers=headers)
    with urllib.request.urlopen(req, timeout=15) as r:
        files = json.loads(r.read().decode())

    # 4. Analyser chaque fichier de code
    code_extensions = (".py", ".js", ".ts", ".java", ".cpp")
    scores = []
    for file_info in files[:20]:  # Limiter a 20 fichiers
        if not file_info["name"].endswith(code_extensions):
            continue
        with urllib.request.urlopen(file_info["download_url"]) as r:
            code = r.read().decode("utf-8", errors="replace")
        features = extract_all_features(code)
        vector = features_to_vector(features)
        proba = _MODEL.predict_proba(np.array([vector]))[0]
        scores.append(float(proba[1]))

    avg_score = sum(scores) / len(scores) if scores else 0.0
    return {"avg_score_ia": avg_score, "files_analyzed": len(scores)}
```

Listing 6 – Ajouter la route d'analyse GitHub dans api.py

6 Étape 5 : Apprentissage Continu

6.1 Principe de la Boucle d'Apprentissage

Le modèle ne doit pas être figé. La boucle entière fonctionne ainsi :

1. **Prédire** : Le modèle analyse un code et donne un score.
2. **Stocker** : La prédiction est sauvegardée dans `predictions.jsonl`.
3. **Auto-étiqueter** : Si le score est très clair (ex : >50%), le système l'enregistre automatiquement comme échantillon valide dans `human_decisions.jsonl`.
4. **Ré-entraîner** : Quand suffisamment de nouvelles prédictions sont accumulées, lancez le script de ré-entraînement. Le modèle génère une nouvelle version `model_v2.pkl` plus précise.

6.2 Script de Ré-entraînement (`training.py`)

```
import json
import numpy as np
import pickle
from pathlib import Path
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split

def run_training(min_samples=1, version="v2"):
    """Lance le re-entrainement a partir des decisions sauvegardees."""
```

```python
    decisions_path = Path("storage/human_decisions.jsonl")

    if not decisions_path.exists():
        return {"error": "Aucun fichier de decisions trouve"}

    # 1. Charger les predictions originales pour retrouver le code
    predictions = {}
    pred_path = Path("storage/predictions.jsonl")
    if pred_path.exists():
        with open(pred_path) as f:
            for line in f:
                entry = json.loads(line)
                predictions[entry["submission_id"]] = entry

    # 2. Charger les decisions humaines (les "corrections")
    X, y = [], []
    with open(decisions_path) as f:
        for line in f:
            d = json.loads(line)
            sid = d["submission_id"]
            label = d["human_label"]
            if sid in predictions:
                code = predictions[sid]["code"]
                from src.features import extract_all_features,
                    features_to_vector
                feats = extract_all_features(code)
                X.append(features_to_vector(feats))
                y.append(label)

    if len(X) < min_samples:
        return {"error": f"Pas assez d'echantillons: {len(X)} < {min_samples}"}

    # 3. Entrainer un nouveau modele
    X_arr, y_arr = np.array(X), np.array(y)
    X_train, X_test, y_train, y_test = train_test_split(
        X_arr, y_arr, test_size=0.2, random_state=42
    )
    clf = RandomForestClassifier(n_estimators=100, max_depth=10, random_state
        =42)
    clf.fit(X_train, y_train)
    score = clf.score(X_test, y_test)

    # 4. Sauvegarder la nouvelle version
    Path("models").mkdir(exist_ok=True)
    with open(f"models/model_{version}.pkl", "wb") as f:
        pickle.dump(clf, f)

    return {"version": version, "accuracy": score, "n_samples": len(y)}
```

Listing 7 – src/training.py - Re-entrainement periodique

7 Étape 6 : Déploiement sur Hugging Face Spaces

7.1 Pourquoi Hugging Face Spaces ?

Hugging Face Spaces est une plateforme gratuite qui héberge des applications web liées à l'IA. Elle supporte nativement Docker et Python, et dispose de serveurs puissants. C'est la solution idéale pour ce type de projet.

7.2 Procédure de Déploiement

7.2.1 1. Créer un Space

1. Allez sur huggingface.co/spaces.
2. Cliquez sur "Create new Space".
3. Choisissez le nom, mettez le SDK sur "Docker" (ou "Gradio" si vous avez une interface Gradio).
4. Laissez la visibilité sur "Public".

7.2.2 2. Fichier de configuration Hugging Face

Créez un fichier `README.md` à la racine du projet avec les métadonnées HF :

```
---
title: Mon Modele Anti-Triche
emoji:
colorFrom: blue
colorTo: purple
sdk: docker
app_port: 7860
---
```

Listing 8 – README.md - En-tete pour Hugging Face

7.2.3 3. Dockerfile

```
FROM python:3.11-slim

WORKDIR /app
COPY . .

RUN pip install --no-cache-dir -r requirements.txt

EXPOSE 7860

CMD ["uvicorn", "src.api:app", "--host", "0.0.0.0", "--port", "7860"]
```

Listing 9 – Dockerfile

7.2.4 4. Ajouter les Secrets (Variables d'environnement)

Dans les paramètres de votre Space (onglet Settings → Variables and secrets) :
— **GITHUB_TOKEN** : Token GitHub Personal Access Token pour éviter les limites de l'API (5000 req/h au lieu de 60).
— **HF_TOKEN** : Token Hugging Face pour que le Space puisse se push ses propres mises à jour.
— **HF_REPO** : L'identifiant de votre Space (ex : `VotreNom/mon-anticheat`).

7.2.5 5. Déployer

```
# Ajouter le remote Hugging Face
git remote add space https://huggingface.co/spaces/VotreNom/MonSpace

# Pousser le code
git push space main
```

Le serveur va automatiquement compiler le Docker, installer les dépendances et lancer l'application. Votre API sera disponible en quelques minutes sur `https://VotreNom-MonSpace.hf.space`.

8 Ajouter des Features Supplémentaires

Pour créer un modèle encore plus précis, voici des features avancées que vous pouvez ajouter à votre `features.py` :

8.1 Features Avancées Suggérées

Feature	Description
`docstring_ratio`	Ratio fonctions avec docstring vs total. L'IA documente systématiquement.
`type_hint_ratio`	Ratio de paramètres avec type hints (`def foo(x: int) -> str`). L'IA utilise toujours les types.
`entropy_variable_names`	Entropie de Shannon des noms de variables. Humain = entropie plus haute (noms variés).
`magic_number_count`	Nombre de nombres "magiques" directs (ex : `if x >42`). L'humain en laisse plus.
`error_handling_density`	Ratio de blocs `try/except` dans le code. L'IA a tendance à en mettre partout.

9 Conclusion et Conseils

Points Clés à Retenir

1. La **qualité des features** prime sur le choix de l'algorithme. Commencez par bien choisir vos indicateurs.
2. Le **dataset initial** doit être équilibré : autant d'exemples Humains que d'exemples IA.
3. La **précision augmente avec le temps** grâce à la boucle d'apprentissage continu.
4. Ne jamais utiliser le modèle seul comme juge final : il doit être un **outil d'aide** à la décision humaine.
5. Pour étendre à d'autres langages (Java, JS, C++), remplacez le parser `ast` par des parsers dédiés (`javalang`, `esprima` pour JS, ou `tree-sitter` universel).

Ce projet complet est disponible et fonctionnel sur Hugging Face Spaces. Il sert de base solide pour tout développeur souhaitant créer son propre outil d'analyse de code source. La pipeline complète (Features → ML → API → Frontend → Cloud) est identique pour n'importe quel domaine de classification, ce qui rend ce guide réutilisable bien au-delà de la détection de triche.